# Guía de entrenamiento local de analítica predictiva

## Objetivo

Esta guía explica cómo los datos del inventario llegan al modelo, cómo entrenar la primera versión local y cómo comprobar si una predicción está habilitada. El proceso no envía ventas ni datos de clientes a servicios externos.

## Requisitos previos

- PHP 7.4 o superior con `curl` y `pdo_mysql`.
- Python 3.10 o superior y las dependencias de `microservicio/requirements.txt`.
- FastAPI con acceso local a `127.0.0.1:8000`.
- Un usuario autenticado con permiso de consulta de Productos; el permiso de modificación es necesario para entrenar.
- Un secreto aleatorio compartido entre el entorno de PHP/Apache y FastAPI. No guardes ese valor en Git.

## 1. Configurar y ejecutar FastAPI

Configura en el entorno que ejecuta PHP:

```
ANALITICA_PREDICTIVA_URL=http://127.0.0.1:8000
ANALITICA_PREDICTIVA_TOKEN=<secreto-aleatorio-privado>
```

Configura exactamente el mismo valor para FastAPI:

```
ANALITICA_PREDICTIVA_TOKEN=<secreto-aleatorio-privado>
```

El token debe tener al menos 32 caracteres. Puedes generar uno con:

```
python -c "import secrets; print(secrets.token_urlsafe(32))"
```

En Windows, define las variables para Apache/PHP en el entorno del servicio o mediante `SetEnv` en la configuración local de Apache; reinicia Apache después del cambio. No guardes el secreto en un archivo versionado. Para una prueba local, define el token también en la sesión de PowerShell donde iniciarás FastAPI:

```
$env:ANALITICA_PREDICTIVA_TOKEN = "pega-aqui-el-token-generado"
```

Desde la carpeta `microservicio`, inicia el servidor enlazado solo a loopback:

```
uvicorn controlador.api_recepcion:app --host 127.0.0.1 --port 8000
```

Mantén esta terminal activa. El servicio existente de recepción y la analítica comparten esta aplicación FastAPI.



## 2. Entender el recorrido de los datos

Sigue este orden al leer el código:

1. **Controles de la página:** `Vista/VistaNew/producto.php` contiene los selectores, el botón Analizar, el botón Entrenar, los KPI, el gráfico y la tabla de valoraciones.
2. **Solicitud del navegador:** la función JavaScript `postAnalytics` envía una acción al URL de la página PHP. El navegador no conoce el token del microservicio.
3. **Permisos y filtros:** `Modelo/Controlador/producto.php` valida permisos, producto/categoría, frecuencia y horizonte. El entrenamiento requiere permiso de modificación.
4. **Consulta a MariaDB:** `Productos::obtenerDatosAnaliticaPredictiva` lee productos activos y líneas de facturas con estado `Pagada` o `Pagada Presencialmente`, limitadas a los últimos 365 días.
5. **Proxy servidor a servidor:** `AnaliticaPredictivaProxy` agrega el token y envía el JSON a `/api/v1/analitica/dashboard`, `/entrenar` o `/feedback`.
6. **Validación FastAPI:** `controlador/api_analitica.py` exige token válido, valida el esquema y llama al módulo de analítica.
7. **Entrenamiento:** `modelo/analitica_predictiva.py::train_models` agrupa las ventas en semanas completas y aprende un modelo Ridge autoregresivo por producto y categoría. Usa las cuatro semanas previas para estimar la siguiente.
8. **Persistencia local:** los coeficientes y métricas se escriben atómicamente en `microservicio/analitica_predictiva/modelos/demanda.json`. Ese archivo se genera en ejecución y está excluido de Git.
9. **Inferencia y presentación:** `build_dashboard` carga el modelo, proyecta el horizonte solicitado, construye los límites de error y deriva los KPI. La respuesta vuelve por PHP y JavaScript actualiza el gráfico/tabla sin recargar el inventario.
10. **Retroalimentación:** las valoraciones se añaden a `microservicio/logs_metricas/feedback_analitica.jsonl`. Su promedio por tipo ajusta la prioridad de recomendaciones siguientes.

## 3. Entrenar el modelo desde Productos

1. Inicia sesión y abre **Gestión de Productos**.
2. En el bloque **Asistente inteligente de inventario**, pulsa **Entrenar modelo local**.
3. PHP vuelve a leer el histórico de 365 días y lo envía al servicio interno.
4. El servicio devuelve la versión y la fecha del modelo, cuántos modelos pudo entrenar y cuántos productos no alcanzaron historial suficiente.
5. Selecciona un producto/categoría y pulsa **Analizar**. El primer entrenamiento puede no producir pronósticos para todos los productos.

Se requiere un mínimo de **24 semanas de ventana** y al menos **seis semanas con ventas** para crear un modelo. Las semanas sin ventas se conservan como cero para que el modelo no confunda semanas sin actividad con datos ausentes. Si se entrena al menos un modelo, se reemplaza el artefacto local; si ninguno alcanza el mínimo, el artefacto previo se conserva y el servicio informa el resultado.

## 4. Leer estados y resultados



- **Ventas reales:** histórico agregado desde facturas aceptadas por la regla de estados.
- **Demanda estimada:** se muestra solo si existe un artefacto entrenado para ese producto/categoría.
- **Insuficiente historial:** menos de seis semanas con ventas dentro de la ventana requerida.
- **Modelo pendiente:** hay datos, pero todavía no se entrenó un modelo para ese alcance.
- **Sin datos:** el producto/categoría no está habilitado o no tiene ventas históricas para mostrar.
- **No disponible:** el KPI depende de un campo nulo. No se cambia `NULL` a cero para aparentar una medición.
- **Banda del gráfico:** límites ilustrativos basados en el error residual del entrenamiento; no son garantía estadística.
- **Compra sugerida:** demanda del horizonte + stock mínimo - stock actual, con piso cero. No descuenta órdenes abiertas.
- **Rotación:** demanda del horizonte dividida por inventario promedio estimado durante ese mismo horizonte.
- **Confianza del combo:** proporción observada de facturas que contienen también el otro producto. Solo se sugieren parejas observadas en dos o más facturas.

## 5. Validar que el entrenamiento evolucionó

Después de cada entrenamiento, confirma:

1. La respuesta del botón muestra la cantidad de modelos y la versión `ridge-lag4-v1`.
2. El archivo local `microservicio/analitica_predictiva/modelos/demanda.json` tiene una nueva fecha `trained_at`.
3. Al analizar un producto con historia suficiente, el estado pasa de `modelo_pendiente` a `disponible`.
4. El gráfico mantiene el histórico real y añade fechas futuras, línea de demanda estimada y banda.
5. Un producto sin historial suficiente mantiene vacíos sus valores estimados; no debe presentar ceros ficticios.

Las pruebas automatizadas del módulo se ejecutan desde la raíz del repositorio:

```
python -m unittest microservicio.tests.test_analitica_predictiva -v
```

## 6. Valorar recomendaciones

En la tabla, selecciona de una a cinco estrellas para una recomendación. El servicio guarda el tipo, identificador, estrellas, usuario autenticado y fecha. En consultas posteriores, la última valoración de cada usuario para una recomendación se incorpora al promedio de su tipo (`combo` o `reposicion`) y modifica la prioridad mostrada.

Esta señal representa preferencia del usuario, no resultado comercial. Para medir efectividad de una promoción se necesitaría guardar también si el combo se creó, cuándo se publicó y cuántas unidades se vendieron después.

## 7. Problemas frecuentes



| Mensaje o síntoma | Comprobación |
|---|---|
| Servicio no disponible | FastAPI está iniciado y escucha en `127.0.0.1:8000`. |
| Token inválido / analítica no configurada | Ambas aplicaciones leen el mismo `ANALITICA_PREDICTIVA_TOKEN`; reinicia Apache después de cambiar el entorno. |
| cURL no disponible | Habilita la extensión `curl` en el `php.ini` usado por Apache. |
| Modelo pendiente | Pulsa Entrenar con un usuario que tenga permiso de modificación. |
| Historial insuficiente | Se necesitan al menos seis semanas con ventas dentro de la ventana de entrenamiento de 24 semanas. |
| KPI «No disponible» | Revisa si stock o stock mínimo está en `NULL`; el sistema no inventa el valor. |
| No se recomiendan combos | La regla requiere que la pareja aparezca junta en al menos dos facturas pagadas. |

## Seguridad y operación

Mantén FastAPI en loopback cuando PHP comparte el mismo host. Usa el secreto por entorno, no por código ni en JavaScript. Conserva los archivos del modelo y de valoraciones fuera de Git; respáldalos según la política local de la empresa. El endpoint de entrenamiento escribe un único artefacto, así que ejecuta FastAPI con un solo worker si se activa entrenamiento concurrente.